Apollo AgriVerse: Execution Manual & Roadmap

Clear Plain-English Algorithm Guide & Code Implementation | Grape Cultivation Scope

1. Executive Overview & Scope Focus

This document provides a fully clear, step-by-step walkthrough of how **Apollo AgriVerse** works. For our active phase, we focus specifically on **Grape Cultivation (*Vitis vinifera*)** across Maharashtra districts (Nashik, Sangli, Solapur, Pune, Ahmednagar, Satara, Osmanabad, Latur, Ratnagiri, Kolhapur).

Key Architecture Rule: All data tables use a standard crop_id (e.g., 'crop_grape'). This means the entire system is built so that adding new crops in the future requires zero database code rewrites.

2. KnowledgeBase Dataset Schemas (5 Core CSVs)

Our Suitability Engine reads 5 simple CSV files located inside 02_Datasets/KnowledgeBase/:

File Name	What It Stores (Simple Terms)	Key Field Columns	Status
01_crop_db.csv	Overall grape plant needs (temperature, pH range, salinity limits)	crop_id, crop_name, min_temp_c, max_temp_c, ph_min, ph_max	Team Assigned
02_crop_variety_db.csv	Specific grape types (Thompson Seedless, Tas-A-Ganesh, Flame) & hydrogel dosage	variety_id, crop_id, variety_name, hydrogel_dosage_kg_ha, heat_tolerance_index	Team Assigned
03_soil_db.csv	5 Maharashtra soil types (Black Cotton, Red Loam, Alluvial, Lateritic, Saline)	soil_id, soil_name, texture_class, avg_ph, avg_ec_ds_m	Team Assigned
04_crop_soil_req.csv	Match scores between each grape variety and each soil type	variety_id, soil_id, compatibility_score, yield_potential_factor	Team Assigned
05_region_climate_db.csv	District baseline climate data (Nashik, Sangli, Solapur weather & default soil)	district, state, avg_annual_temp_c, avg_annual_rain_mm, dominant_soil_id	**Completed**

3. Step-by-Step Plain-English Algorithm Walkthrough

Here is exactly how the algorithm works in simple terms, step by step, when a farmer or user uses our app:

Step 1: Get User Inputs (Location & Soil Details)

The user opens the app and selects their **District** (e.g., *Nashik*), their **Soil Type** (e.g., *Black Cotton Soil*), and enters their **Farm Size** (e.g., *2.5 Hectares*). The app sends this data to our Python backend.

Step 2: Fetch Local Weather & Filter Grape Varieties

The system opens 05_region_climate_db.csv to get average weather for Nashik. Next, it looks into 04_crop_soil_req.csv and picks out all grape varieties that can grow in Black Cotton Soil (like Thompson Seedless, Tas-A-Ganesh, Flame Seedless).

Step 3: Calculate the Suitability Score (0 to 100)

For each candidate grape variety, the system calculates a single composite **Suitability Score** using 3 simple checks:

- **Soil Match (50% weight):** How well does this grape variety like this soil type?
- **Heat Tolerance (30% weight):** Can this grape variety handle local heat waves in Nashik?
- **Yield Potential (20% weight):** What is the expected harvest yield multiplier?

Formula: Final Score = (Soil Match × 0.50) + (Heat Score × 0.30) + (Yield Factor × 0.20)

Step 4: Calculate Hydrogel Requirements for the Farm

Different grape varieties need different water retention support. The system reads the required hydrogel dosage per hectare from 02_crop_variety_db.csv and calculates total hydrogel needed:

Total Hydrogel (kg) = Farm Size (in Hectares) × Recommended Dosage (kg/ha)

Step 5: Rank Results & Return Recommendation Payload

The system sorts the grape varieties from highest score to lowest score. The top ranked variety is recommended as the **Best Match**. It packages these results into a simple JSON response for the frontend.

Step 6: Display on Dashboard & Feed Digital Twin

The React web dashboard displays the recommended grape varieties with visually easy cards, progress bars for scores, and exact hydrogel dosage instructions. This data also initializes the Digital Twin simulation model.

4. Concrete Python Code Implementation

Below is the exact Python implementation for the algorithm (05_Backend/suitability_engine.py). It is cleanly structured, commented, and easy for any team member to follow:

```
import pandas as pd def calculate_grape_suitability(district_name, soil_id, farm_size_ha):
    # 1. Load KnowledgeBase CSVs
    varieties_df = pd.read_csv('02_Datasets/KnowledgeBase/02_crop_variety_db.csv')
    requirements_df = pd.read_csv('02_Datasets/KnowledgeBase/04_crop_soil_req.csv')
    climate_df = pd.read_csv('02_Datasets/KnowledgeBase/05_region_climate_db.csv')
    # 2. Filter varieties matching user's soil type
    soil_matches = requirements_df[requirements_df['soil_id'] == soil_id]     results = []
    # 3. Calculate scores for each candidate variety     for _, row in soil_matches.iterrows():
        v_id = row['variety_id']         v_info = varieties_df[varieties_df['variety_id'] == v_id].iloc[0]
        soil_score = row['compatibility_score']  # Range: 0 to 100
        heat_score = v_info['heat_tolerance_index'] * 10  # Scale to 100
        yield_score = row['yield_potential_factor'] * 100  # Scale to 100         # Weighted Formula
        final_score = (soil_score * 0.50) + (heat_score * 0.30) + (yield_score * 0.20)
        total_hydrogel_kg = farm_size_ha * v_info['hydrogel_dosage_kg_ha']         results.append({
            'variety_name': v_info['variety_name'],             'suitability_score': round(final_score, 1),
            'recommended_hydrogel_kg': round(total_hydrogel_kg, 1),
            'yield_factor': row['yield_potential_factor']         })
    # 4. Sort from highest score to lowest score
    ranked_results = sorted(results, key=lambda x: x['suitability_score'], reverse=True)
    return ranked_results
```

5. Concrete System Modifications Checklist

System Area	Exact File Modification Needed	Goal	Priority
Datasets	Fill and save CSVs 01 to 04 in 02_Datasets/KnowledgeBase/	Provides complete grape cultivar and soil data.	**CRITICAL**
Backend Logic	Create 05_Backend/suitability_engine.py using code above	Executes suitability score calculations.	**CRITICAL**
API Layer	Add FastAPI endpoint POST /api/v1/recommend	Connects frontend input to Python backend logic.	**HIGH**
Frontend UI	Build District & Soil dropdown form in React	Lets farmers select inputs and view ranked grapes.	MEDIUM

Document Lead: Vish | **Project:** Apollo AgriVerse
All team members must complete assigned database CSV files to enable engine testing.